

Convolución y correlación de series temporales

Del producto de sumas en el tiempo al producto simple en frecuencia — ejemplos en R y Python

Dr. Marcelo Risk

Data Mining de Series Temporales · Maestría en Explotación de Datos y Descubrimiento de Conocimientos · FCEyN UBA · 2026

¿Qué es la convolución de dos series?

Procesar una serie con un sistema lineal (por ejemplo, un filtro)

En el DT

$$x(t) * h(t) = y(t)$$

$$y(n) = \sum x(n-k) \cdot h(k) \quad (k = 0 \dots N-1)$$

La operación en el dominio del tiempo se llama convolución y produce una nueva serie temporal.

En el DF

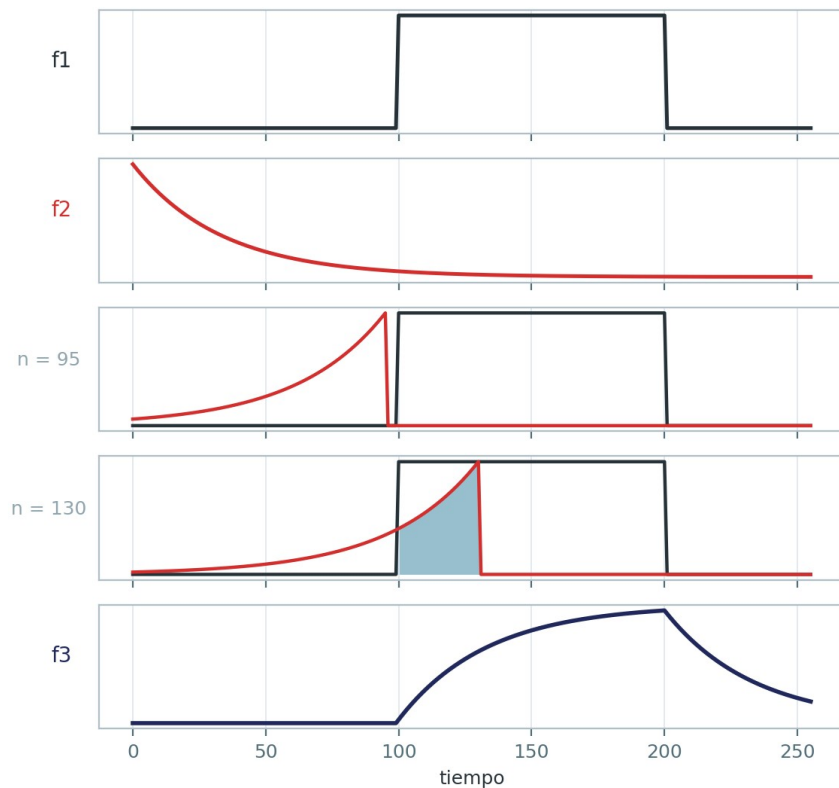
$$X(f) \cdot H(f) = Y(f)$$

producto algebraico simple,
componente a componente

Representada en el dominio de la frecuencia, la misma operación se vuelve una multiplicación de espectros.

Idea central: convolucionar en el tiempo equivale a multiplicar en frecuencia. Esta dualidad DT $\leftrightarrow$ DF es la base del filtrado, la correlación y la modulación.

Cómo funciona: una serie se desliza sobre la otra



Lectura del diagrama

f2 se invierte en el tiempo y se desliza a lo largo de f1.

En cada posición n, la salida es la suma de los productos en la zona de solapamiento (área sombreada).

El resultado f3 crece mientras el solapamiento aumenta y decae exponencialmente cuando el pulso queda atrás.

Señales de prueba: pulso y exponencial

El mismo experimento, lado a lado

R

```
N = 256
tiempo = 0:(N-1)

# f1: pulso
f1 = rep(0, N)
f1[100:200] = 1

# f2: exponencial decreciente
f2 = 1*exp(-0.03*tiempo)

op <- par(mfrow = c(1, 2))
plot(tiempo, f1, type='l')
plot(tiempo, f2, type='l')
```

Python

```
import numpy as np
import matplotlib.pyplot as plt

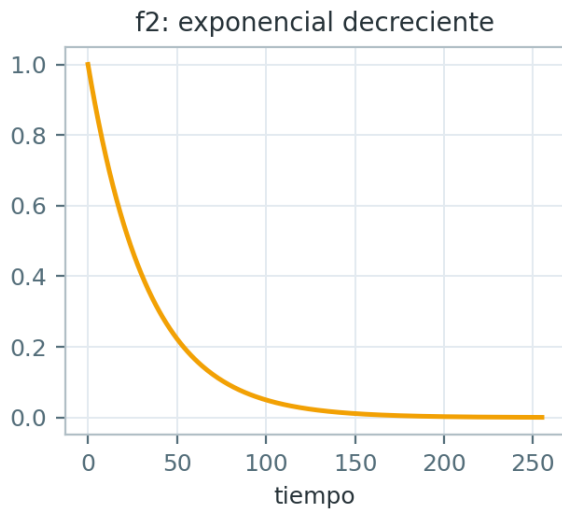
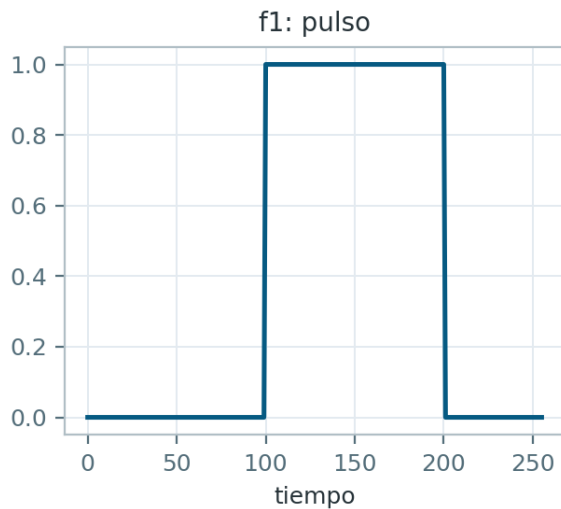
N = 256
t = np.arange(N)

# f1: pulso
f1 = np.zeros(N)
f1[100:201] = 1

# f2: exponencial decreciente
f2 = np.exp(-0.03*t)

fig, ax = plt.subplots(1, 2)
ax[0].plot(t, f1)
ax[1].plot(t, f2)
```

Las señales generadas



Qué esperamos

f1 concentra su energía en un intervalo (100–200).

f2 es la respuesta típica de un sistema de primer orden.

Su convolución debería “arrastrar” el pulso con una cola exponencial.

Convolución en el DT: el algoritmo

La suma de productos, escrita a mano y vectorizada

R

```
f3 = rep(NA, N)
for(i in 1:N)
{
  suma = 0
  j = 1
  while(j <= i)
  {
    suma = suma +
      f1[j]*f2[i-j+1]
    j = j + 1
  }
  f3[i] = suma
}

plot(tiempo, f3,
     type='l', col='red')
```

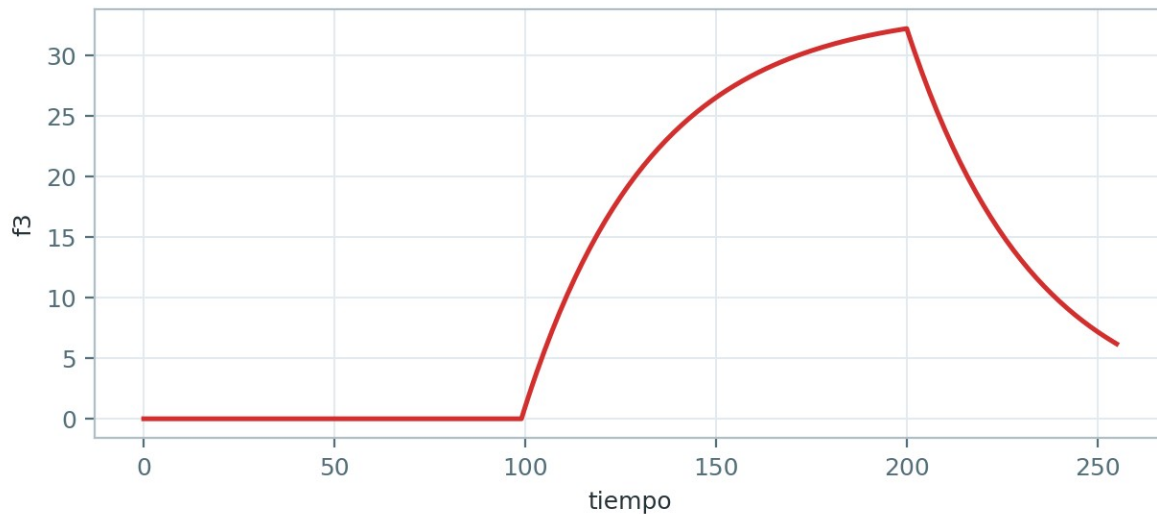
Python

```
f3 = np.zeros(N)
for i in range(N):
    f3[i] = np.sum(
        f1[:i+1] * f2[i::-1])

# equivalente vectorizado:
f3 = np.convolve(f1, f2)[:N]

plt.plot(t, f3, 'r')
```

Resultado: el pulso filtrado por la exponencial



Lectura del gráfico

f_3 arranca en cero: no hay solapamiento hasta la muestra 100.

Crece como carga de un sistema de primer orden mientras dura el pulso.

Al terminar el pulso (muestra 200), descarga con la cola de f_2 .

Convolución en el DF

FFT de cada serie, producto componente a componente y FFT inversa

R

```
fft.f1 = fft(f1)
fft.f2 = fft(f2)

fft.f3 = fft.f1 * fft.f2

f3b = Re(fft(fft.f3,
             inverse=TRUE)/N)

plot(tiempo, f3b, type='l')
lines(tiempo, f3, col='red')
```

Python

```
F1 = np.fft.fft(f1)
F2 = np.fft.fft(f2)

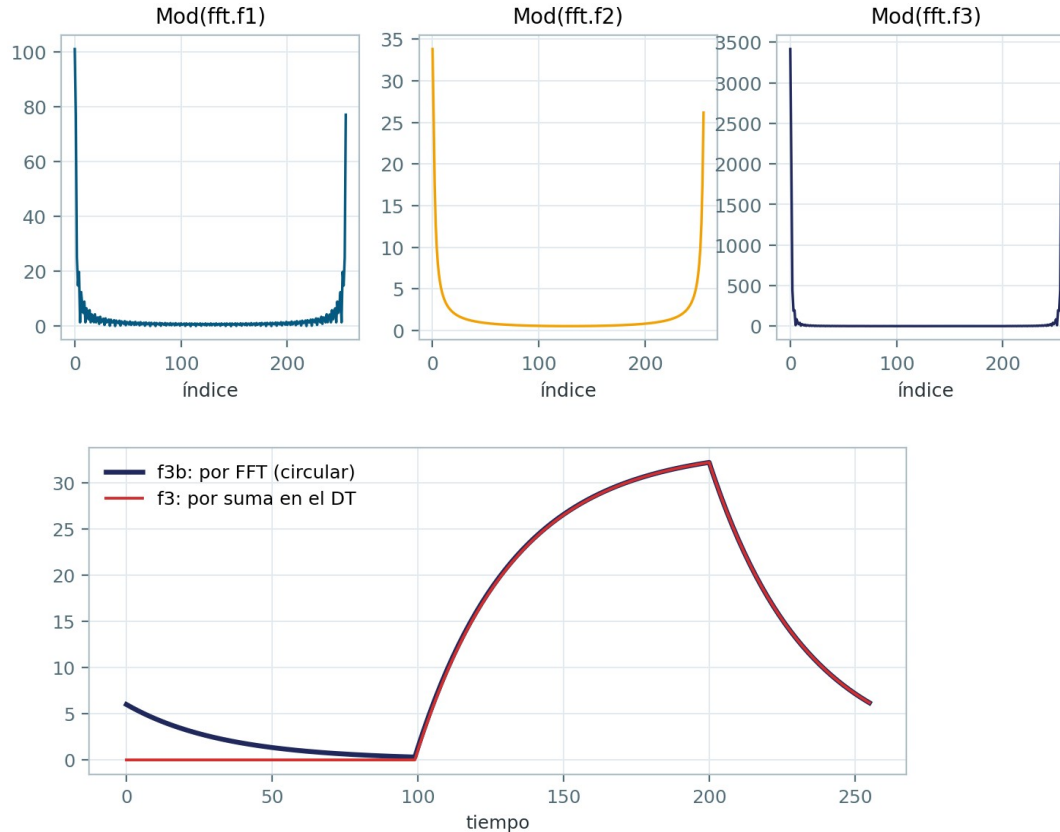
F3 = F1 * F2

f3b = np.real(
    np.fft.ifft(F3))

plt.plot(t, f3b)
plt.plot(t, f3, 'r')
```

Atención: la FFT asume que la serie es periódica → la convolución resultante es circular y puede diferir del cálculo lineal en los bordes.

Verificación: el mismo resultado por dos caminos



Observación

Arriba: los espectros de f1, f2 y su producto fft.f3 .

Abajo: la antitransformada f3b coincide con f3 salvo al comienzo.

Esa cola inicial es el “envoltorio” de la convolución circular: el final de la serie se mezcla con el principio.

Atajo en R: convolve

Un solo argumento separa la convolución de la correlación cruzada

R

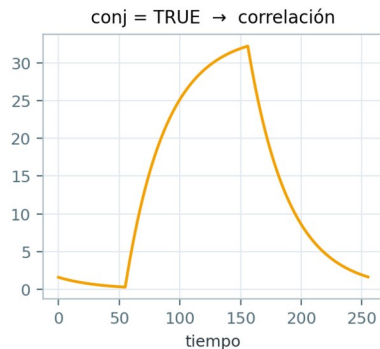
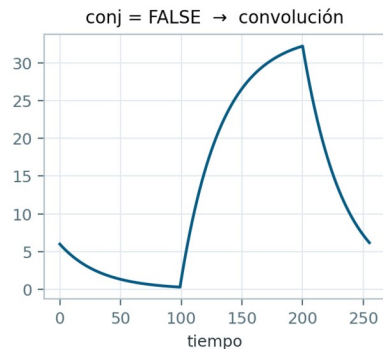
```
# convolución
f1f2 = convolve(f1, f2,
               conj = FALSE)

# correlación cruzada
f1f2 = convolve(f1, f2,
               conj = TRUE)
```

Python

```
# convolución
f1f2 = np.real(np.fft.ifft(
    np.fft.fft(f1) *
    np.fft.fft(f2)))

# correlación cruzada
f1f2 = np.real(np.fft.ifft(
    np.conj(np.fft.fft(f1)) *
    np.fft.fft(f2)))
```



conj = TRUE conjuga la primera FFT

El mismo motor FFT calcula ambas operaciones: $X \cdot Y$ da la convolución y $X^* \cdot Y$ la correlación. Nótese que la correlación no es simétrica: mide el parecido al desplazar una serie sobre la otra.

Correlación cruzada y autocorrelación

No un número, sino una tercera serie temporal

Correlación cruzada

$$r_{xy}(n) = \sum x(n) \cdot y(n+k)$$

en el DF: $r_{xy}(f) = X^*(f) \cdot Y(f)$

Mide la similaridad entre dos series cuando una se desplaza a lo largo de la línea de tiempo.

Autocorrelación

$$r_{xx}(n) = \sum x(n) \cdot x(n+k)$$

en el DF: $r_{xx}(f) = X^*(f) \cdot X(f)$

Similaridad de una serie consigo misma: máxima en el desplazamiento cero, y reveladora en los demás.

A diferencia de la correlación entre dos conjuntos de números (un solo coeficiente), **aquí el resultado es una serie completa: un valor por cada desplazamiento (lag).**

La ccf encuentra el desplazamiento

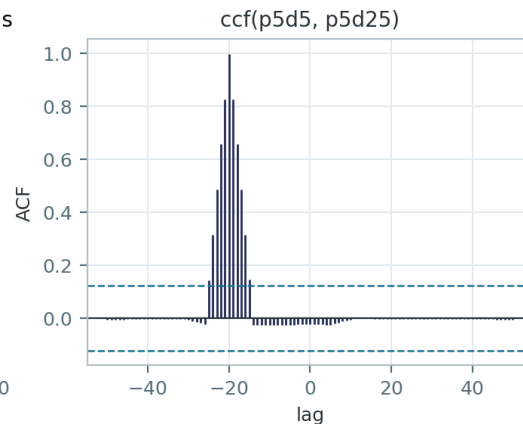
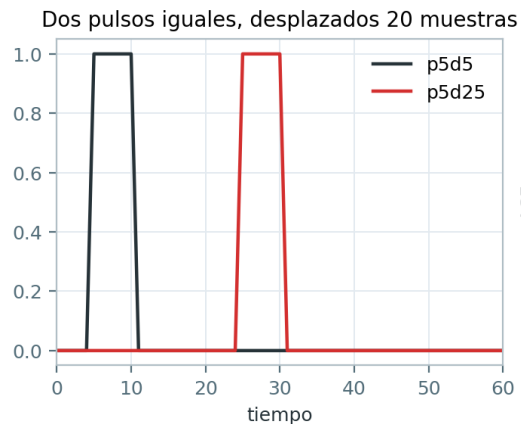
Dos pulsos iguales separados 20 muestras

R

```
p5d5 = rep(0,N)
p5d5[5:10] = 1

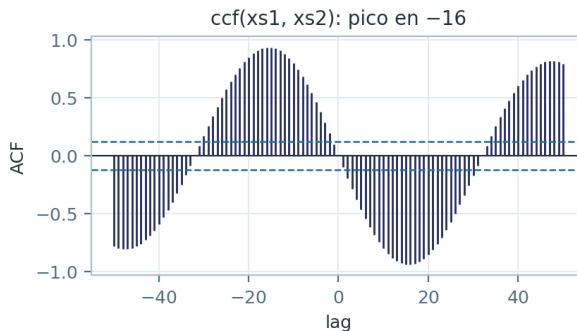
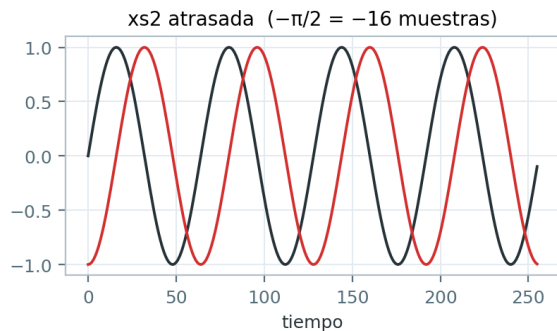
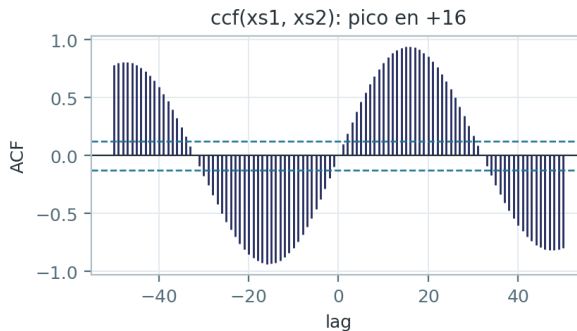
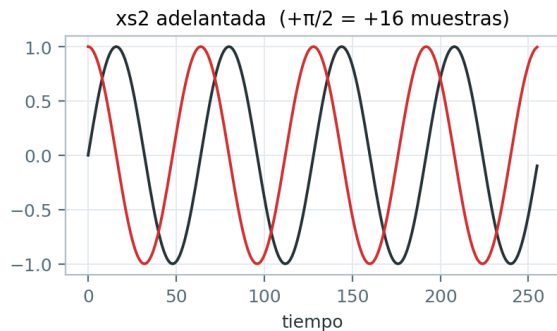
p5d25 = rep(0,N)
p5d25[25:30] = 1

ccf(p5d5, p5d25,
    lag.max = 50)
```



Lectura: el pico de la ccf aparece en lag = -20: hay que retrasar el primer pulso 20 muestras para superponerlo con el segundo. Las líneas punteadas marcan el umbral de significancia.

Senoides desfasadas: el signo del lag delata quién adelanta



Observación

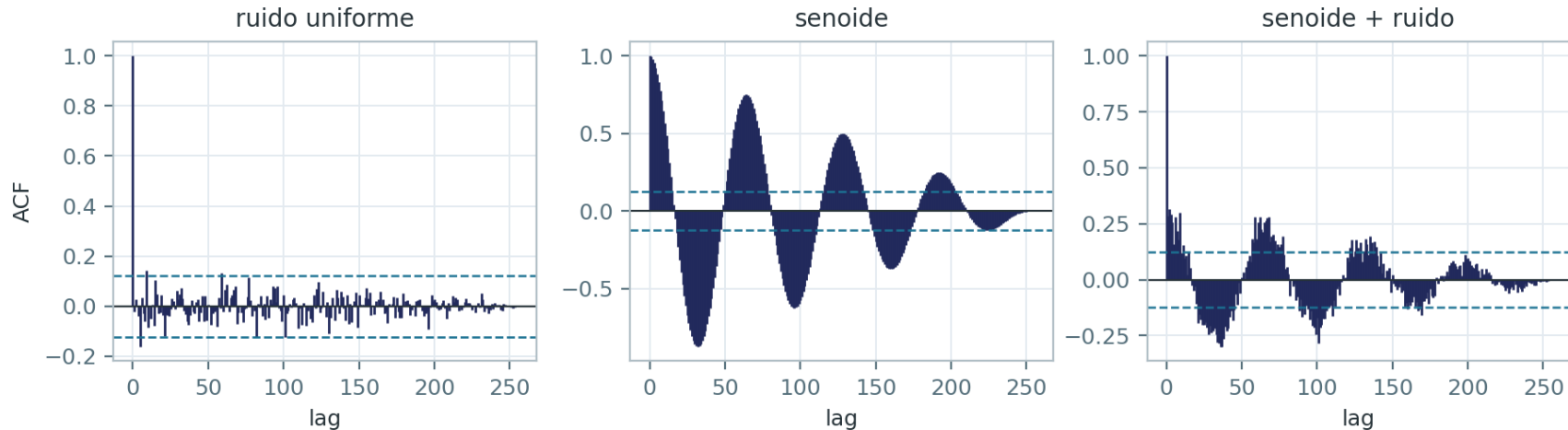
Con $xs2 = \sin(\omega t + \pi/2)$, el máximo de la ccf cae en lag +16: xs2 se adelanta un cuarto de ciclo.

Con fase $-\pi/2$, el pico se espeja a lag -16.

En una senoide de 4 ciclos sobre $N = 256$, $\pi/2$ equivale exactamente a 16 muestras.

Autocorrelación: la firma de cada señal

$acf(x, lag.max = N)$ sobre ruido, senoide y su mezcla



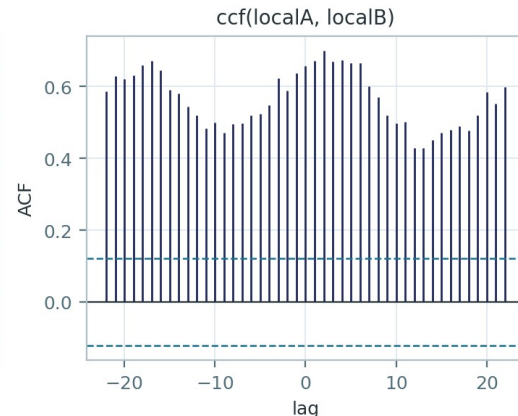
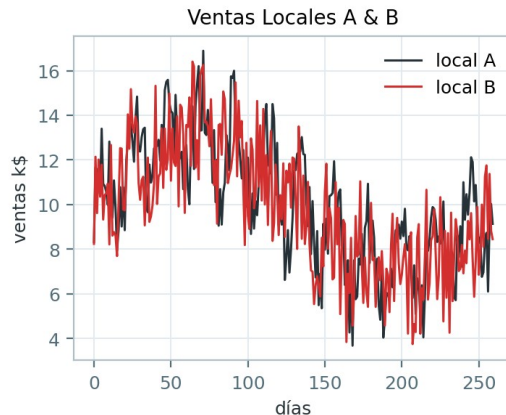
El ruido **solo se parece a sí mismo en lag 0**; la senoide repite su período indefinidamente; la mezcla conserva la periodicidad pero atenuada por el ruido — la acf revela estructura oculta a simple vista.

Ejercicio: ¿las ventas de un local siguen a las del otro?

Un año de ventas diarias con estacionalidad anual y mensual

R

```
dos.locales =  
  read.csv('DosLocales.csv')  
  
plot(dos.locales$t,  
     dos.locales$localA,  
     type='l')  
lines(dos.locales$t,  
      dos.locales$localB,  
      col='red')  
  
ccf(dos.locales$localA,  
    dos.locales$localB)
```



Lectura: la inspección visual no sugiere nada definitivo, pero la ccf muestra correlación significativa en todos los lags (tendencia común) con máximos periódicos: los ciclos mensuales de ambos locales están acoplados.

Ejercicio: radio AM

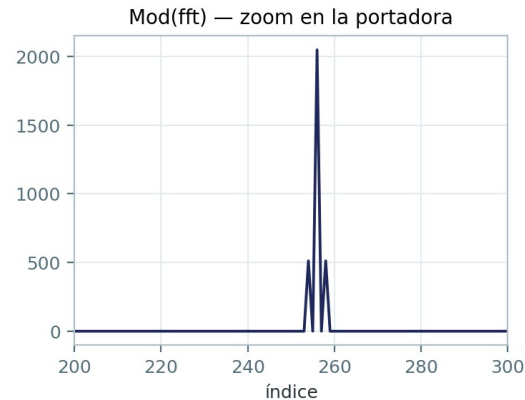
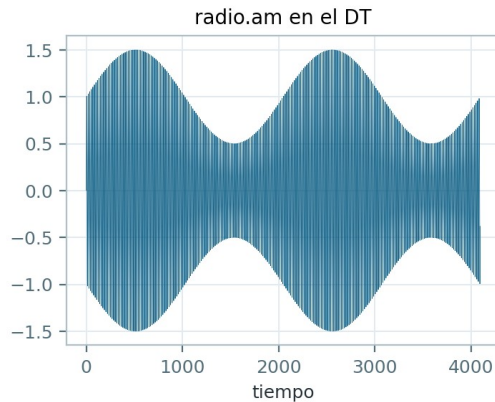
La dualidad al revés: multiplicar en el DT = convolucionar en el DF

R

```
N = 16*256
tiempo = 0:(N-1)
cm = 0.5

portadora =
  sin(256*2*pi*tiempo/N)
moduladora =
  sin(2*2*pi*tiempo/N)

radio.am =
  (1+cm*moduladora) *
  portadora
```



Lectura: multiplicar la portadora por la moduladora convoluciona sus espectros: aparece el pico de la portadora (armónica 256) flanqueado por dos bandas laterales en 254 y 258 — el espectro clásico de AM.

Ideas para llevarse

Dualidad DT $\leftrightarrow$ DF Convolución en el tiempo = producto en frecuencia, y viceversa: multiplicar en el tiempo (AM) convoluciona los espectros.

Correlación = convolución conjugada El mismo motor FFT resuelve ambas: $X \cdot Y$ convoluciona, $X^* \cdot Y$ correlaciona (convolve con `conj=TRUE`).

El lag cuenta la historia El pico de la ccf indica cuánto (y en qué dirección) se desplaza una serie respecto de la otra.

La acf revela estructura El ruido muere en lag 0; la periodicidad sobrevive incluso enterrada en ruido.

R: `convolve` · `ccf` · `acf` · Python: `np.convolve` · `np.fft` · `np.correlate`